CS2023 Algoritmos y Estructuras de Datos
Árboles binarios de búsqueda balanceados
Semana 9

Víctor Racsó Galván Oyola

vgalvan@utec.edu.pe

5 de octubre de 2026

Objetivos de la sesión



Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Por qué un BST necesita balancearse, y por qué no basta con exigir que sea completo

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Por qué un BST necesita balancearse, y por qué no basta con exigir que sea completo
- Rotaciones: cambiar la forma del árbol sin romper el orden

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Por qué un BST necesita balancearse, y por qué no basta con exigir que sea completo
- Rotaciones: cambiar la forma del árbol sin romper el orden
- AVL Tree: factor de balance, altura $O(\lg n)$ y rebalanceo con rotaciones

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Por qué un BST necesita balancearse, y por qué no basta con exigir que sea completo
- Rotaciones: cambiar la forma del árbol sin romper el orden
- AVL Tree: factor de balance, altura $O(\lg n)$ y rebalanceo con rotaciones
- Red-Black Tree: colores, altura $O(\lg n)$ y la idea de su rebalanceo

Objetivos de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Por qué un BST necesita balancearse, y por qué no basta con exigir que sea completo
- Rotaciones: cambiar la forma del árbol sin romper el orden
- AVL Tree: factor de balance, altura $O(\lg n)$ y rebalanceo con rotaciones
- Red-Black Tree: colores, altura $O(\lg n)$ y la idea de su rebalanceo
- Comparar AVL y Red-Black, y el TDA de un árbol balanceado

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El talón de Aquiles del BST (clase 12)

Todas sus operaciones cuestan $O(h)$. Si insertamos $1, 2, \ldots, n$ en orden, el árbol degenera en una cadena: $h = n - 1$.

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El talón de Aquiles del BST (clase 12)

Todas sus operaciones cuestan $O(h)$. Si insertamos $1, 2, \ldots, n$ en orden, el árbol degenera en una cadena: $h = n - 1$.

¿Qué tan grave es eso en la práctica?

Motivación

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El talón de Aquiles del BST (clase 12)

Todas sus operaciones cuestan $O(h)$. Si insertamos $1, 2, \ldots, n$ en orden, el árbol degenera en una cadena: $h = n - 1$.

¿Qué tan grave es eso en la práctica?

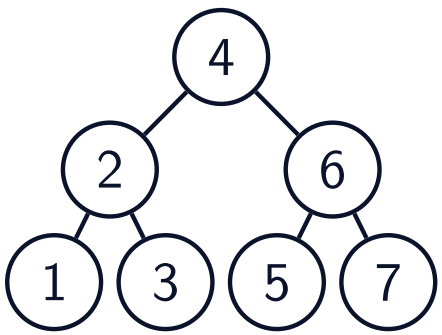
Muy grave

Insertar $n = 10^5$ claves ordenadas cuesta $0 + 1 + \cdots + (n-1) \approx 5 \cdot 10^9$ pasos. Con altura $\lg n \approx 17$ serían unos $1{,}7 \cdot 10^6$. Hoy: cómo **garantizar** $h = O(\lg n)$, sin importar el orden de las operaciones.

¿Qué significa estar balanceado?

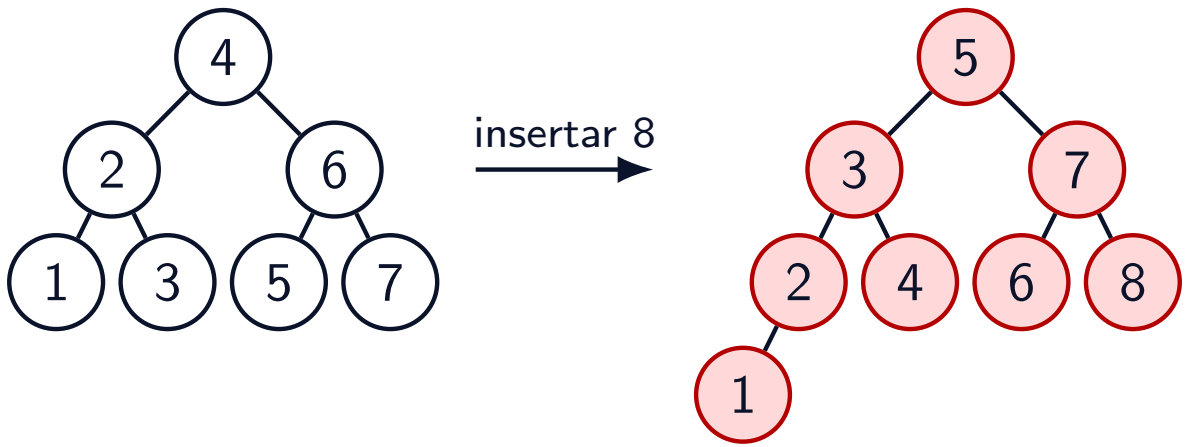
¿Y si exigimos que sea completo?

Un árbol completo tiene la menor altura posible. ¿Por qué no exigir que el BST sea siempre completo, como un heap?



¿Y si exigimos que sea completo?

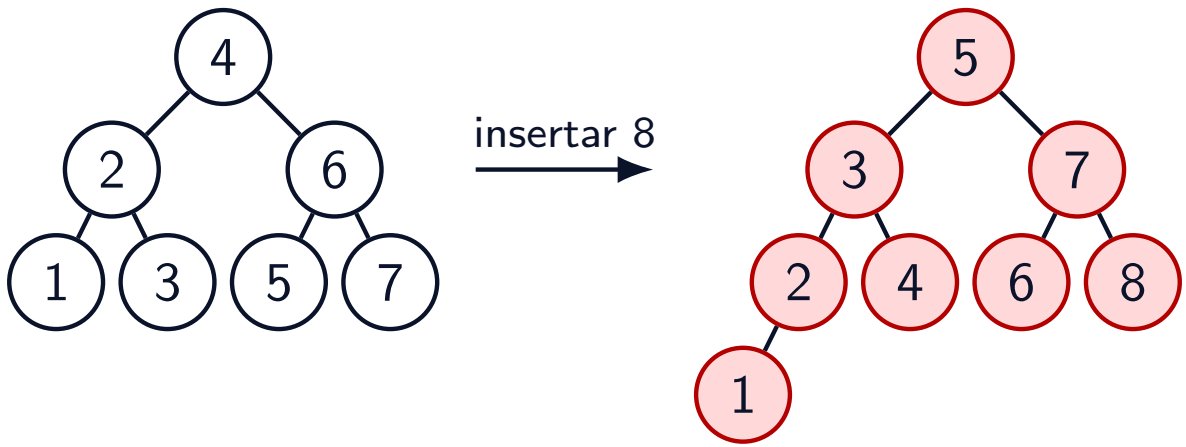
Un árbol completo tiene la menor altura posible. ¿Por qué no exigir que el BST sea siempre completo, como un heap?



Para seguir completo con claves 1.,8, la raíz tiene que pasar a ser 5, y **todas** las posiciones cambian de clave: $\Theta(n)$ trabajo por una sola inserción.

¿Y si exigimos que sea completo?

Un árbol completo tiene la menor altura posible. ¿Por qué no exigir que el BST sea siempre completo, como un heap?



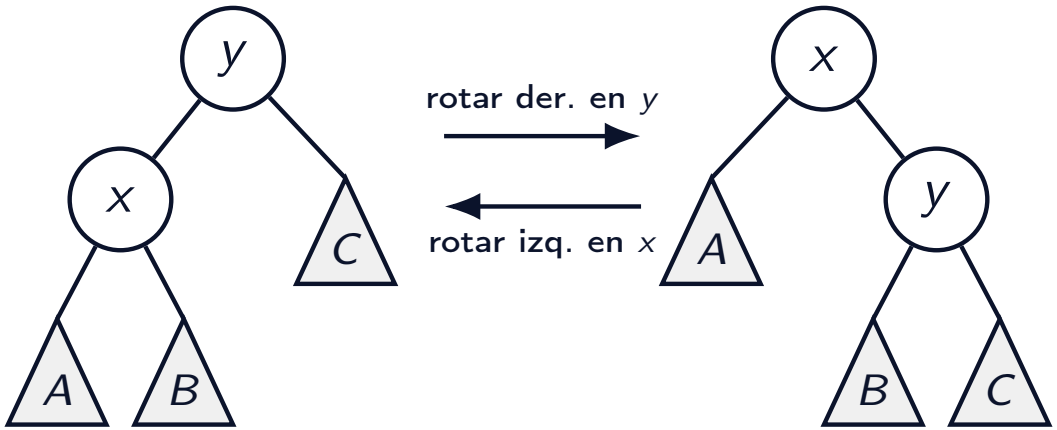
Para seguir completo con claves 1.,8, la raíz tiene que pasar a ser 5, y **todas** las posiciones cambian de clave: $\Theta(n)$ trabajo por una sola inserción.

Hace falta una condición más débil

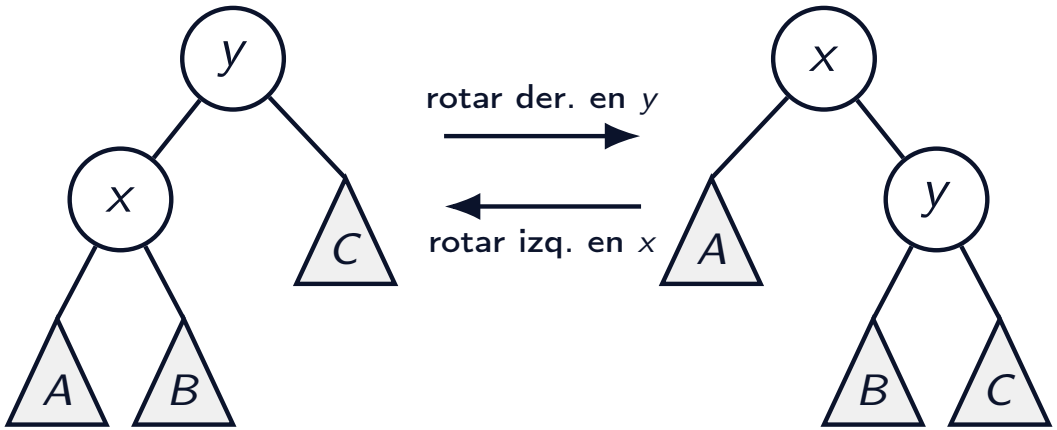
Una que garantice $h = O(\lg n)$ (con alguna constante) y que se pueda restaurar en $O(\lg n)$ tras cada inserción o eliminación.

Rotaciones

Rotación: cambiar la forma sin romper el orden

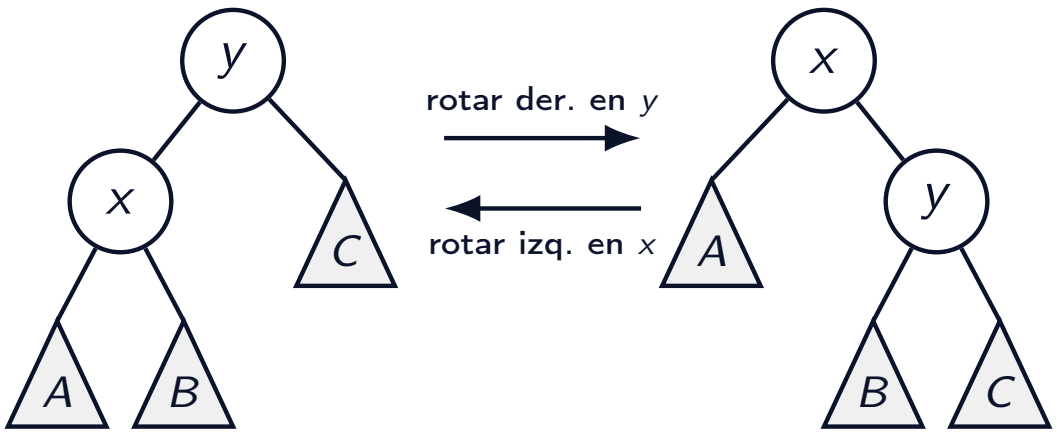


Rotación: cambiar la forma sin romper el orden



El recorrido inorder es A, x, B, y, C en los dos árboles: la propiedad BST se conserva. Solo cambian tres punteros: $O(1)$, sin importar el tamaño de A, B y C.

Rotación: cambiar la forma sin romper el orden



El recorrido inorder es A, x, B, y, C en los dos árboles: la propiedad BST se conserva. Solo cambian tres punteros: $O(1)$, sin importar el tamaño de A, B y C.

Lo que sí cambia: las alturas

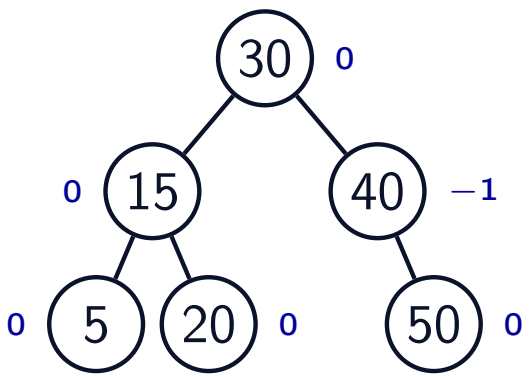
A sube un nivel, C baja uno, y B solo cambia de padre. Una rotación pasa altura de un lado al otro: justo lo que hace falta para corregir un desbalance.

AVL Tree

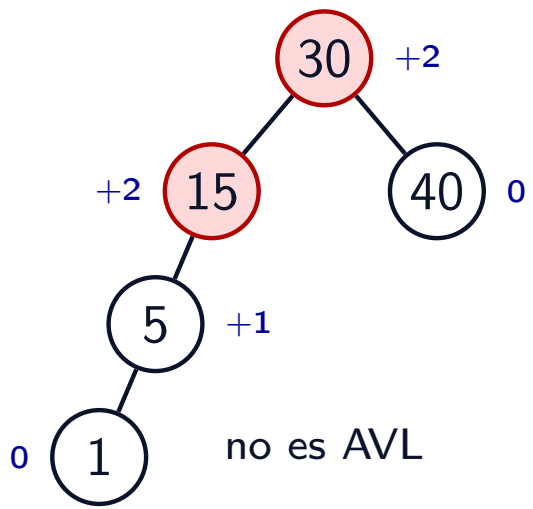
El factor de balance

Definición

Con $h(\text{NULO}) = -1$: $FB(v) = h(v.izq) - h(v.der)$. Un árbol es **AVL** (Adelson-Velsky y Landis, 1962) si $|FB(v)| \leq 1$ en **todo** nodo.



AVL

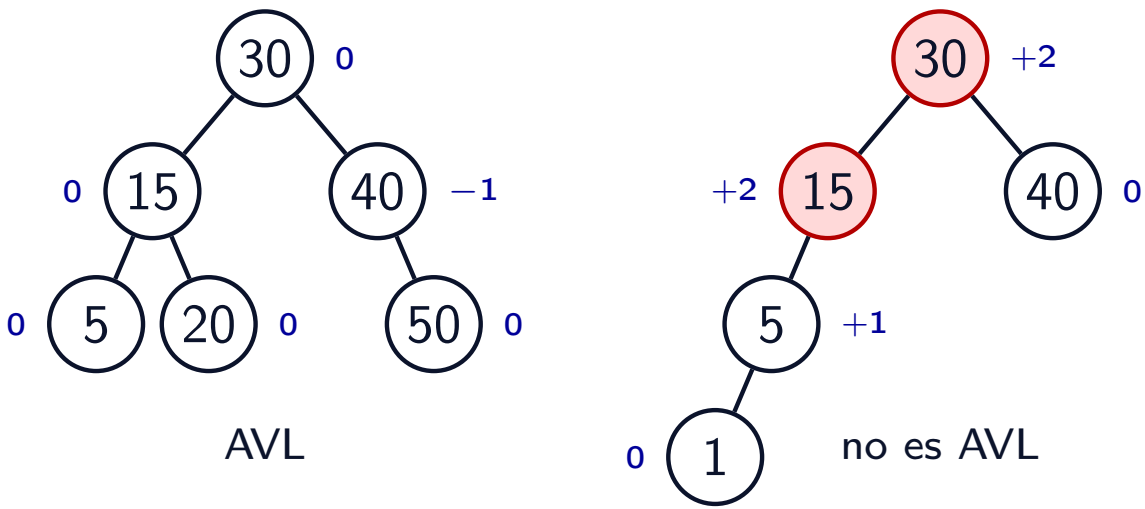


no es AVL

El factor de balance

Definición

Con $h(\text{NULO}) = -1$: $FB(v) = h(v.izq) - h(v.der)$. Un árbol es **AVL** (Adelson-Velsky y Landis, 1962) si $|FB(v)| \leq 1$ en **todo** nodo.



En azul, el factor de balance de cada nodo. A la derecha, 15 tiene a la izquierda un subárbol de altura 1 y a la derecha ninguno (-1): $FB = 1 - (-1) = 2$.

¿Por qué la altura es $O(\lg n)$?

$N(h)$: el mínimo número de nodos de un AVL de altura h

El AVL más "flaco" de altura h tiene una raíz, un subárbol de altura $h-1$ y otro de altura $h-2$, ambos también mínimos:

$$N(h) = 1 + N(h-1) + N(h-2), \qquad N(0) = 1,\ N(1) = 2.$$

¿Por qué la altura es $O(\lg n)$?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$N(h)$: el mínimo número de nodos de un AVL de altura h

El AVL más "flaco" de altura h tiene una raíz, un subárbol de altura $h-1$ y otro de altura $h-2$, ambos también mínimos:

$$N(h) = 1 + N(h-1) + N(h-2), \qquad N(0) = 1,\ N(1) = 2.$$

Una cota sencilla

Como $N(h-1) > N(h-2)$, se tiene $N(h) > 2N(h-2)$: bajando de a dos niveles, $N(h) \geq 2^{\lfloor h/2 \rfloor}$. Si el AVL tiene n nodos, $n \geq N(h) \geq 2^{\lfloor h/2 \rfloor}$, así que $h \leq 2 \lg n + 1$.

¿Por qué la altura es $O(\lg n)$?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

$N(h)$: el mínimo número de nodos de un AVL de altura h

El AVL más "flaco" de altura h tiene una raíz, un subárbol de altura $h-1$ y otro de altura $h-2$, ambos también mínimos:

$$N(h) = 1 + N(h-1) + N(h-2), \qquad N(0) = 1,\ N(1) = 2.$$

Una cota sencilla

Como $N(h-1) > N(h-2)$, se tiene $N(h) > 2N(h-2)$: bajando de a dos niveles, $N(h) \geq 2^{\lfloor h/2 \rfloor}$. Si el AVL tiene n nodos, $n \geq N(h) \geq 2^{\lfloor h/2 \rfloor}$, así que $h \leq 2 \lg n + 1$.

Con más cuidado (es casi la recurrencia de Fibonacci) sale $h < 1{,}45 \lg(n+2)$: un AVL es casi tan bajo como un árbol perfecto.

Insertar en un AVL: ¿qué se rompe?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Primero, insertar como en un BST

El nodo nuevo queda como hoja: solo cambian las alturas de sus **ancestros**, y cada una a lo más en $+1$.

Insertar en un AVL: ¿qué se rompe?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Primero, insertar como en un BST

El nodo nuevo queda como hoja: solo cambian las alturas de sus **ancestros**, y cada una a lo más en $+1$.

¿Dónde se rompe el balance, si se rompe?

Insertar en un AVL: ¿qué se rompe?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Primero, insertar como en un BST

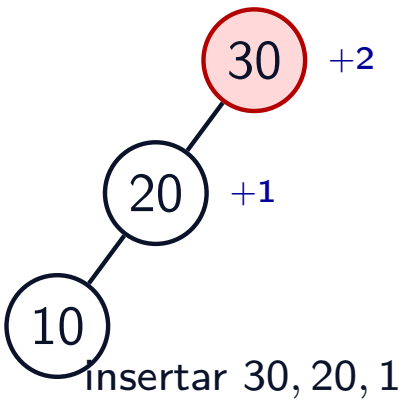
El nodo nuevo queda como hoja: solo cambian las alturas de sus **ancestros**, y cada una a lo más en $+1$.

¿Dónde se rompe el balance, si se rompe?

En algún ancestro, que queda con $|FB| = 2$

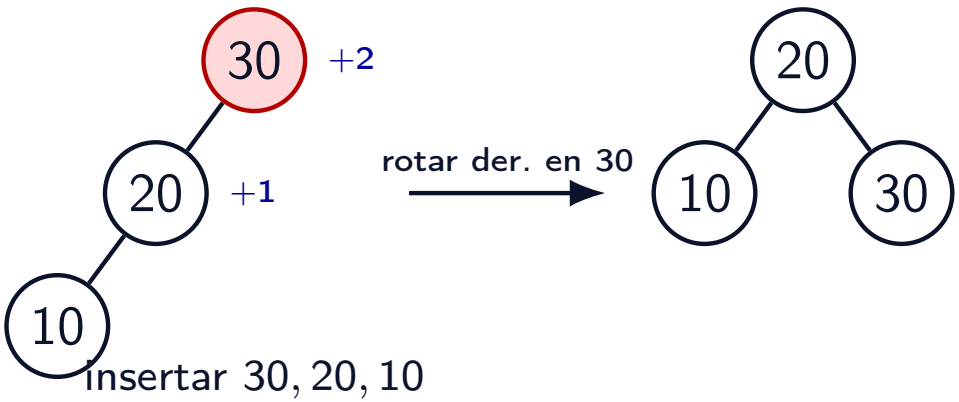
Subiendo desde el nodo nuevo, el **primer** ancestro desbalanceado se arregla con una o dos rotaciones, según por dónde bajó la inserción desde él: izquierda-izquierda (LL), derecha-derecha (RR), izquierda-derecha (LR) o derecha-izquierda (RL).

Casos LL y RR: una rotación



LL: el desbalance está en el hijo izquierdo, y la inserción siguió por **su** izquierda.

Casos LL y RR: una rotación

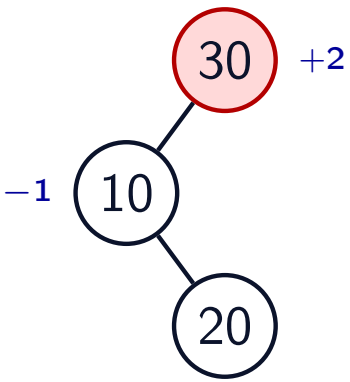


LL: el desbalance está en el hijo izquierdo, y la inserción siguió por **su** izquierda.

Una rotación a la derecha en el nodo desbalanceado

Todos quedan con $FB = 0$. RR es el espejo: insertar $10, 20, 30$ se arregla con una rotación a la **izquierda** en 10.

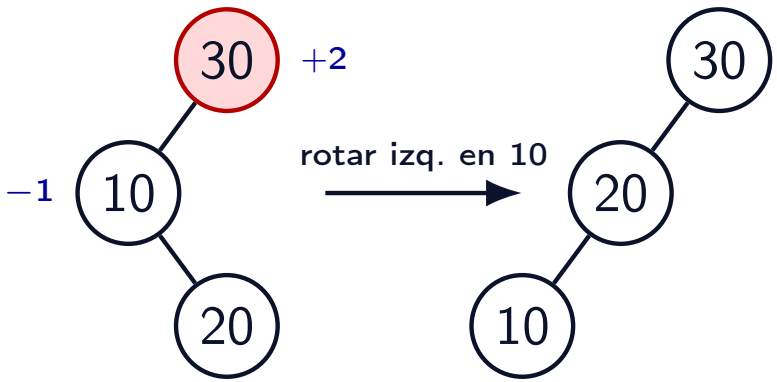
Casos LR y RL: dos rotaciones



insertar $30, 10, 20$

LR: el desbalance está en el hijo izquierdo, pero la inserción siguió por **su** derecha. Una sola rotación en 30 no alcanza: dejaría el mismo problema reflejado, con 10 arriba y $FB = -2$.

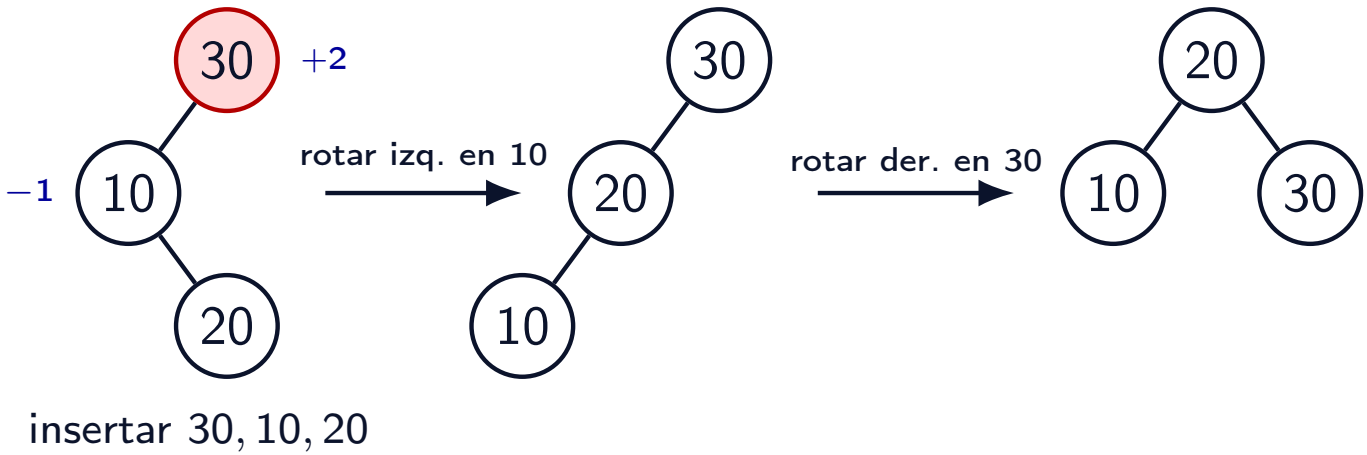
Casos LR y RL: dos rotaciones



insertar $30, 10, 20$

LR: el desbalance está en el hijo izquierdo, pero la inserción siguió por **su** derecha. Una sola rotación en 30 no alcanza: dejaría el mismo problema reflejado, con 10 arriba y $FB = -2$.

Casos LR y RL: dos rotaciones

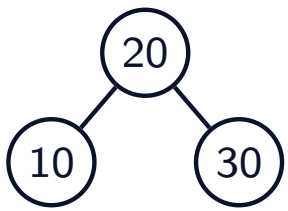


LR: el desbalance está en el hijo izquierdo, pero la inserción siguió por **su** derecha. Una sola rotación en 30 no alcanza: dejaría el mismo problema reflejado, con 10 arriba y $FB = -2$.

Primero rotar en el hijo, luego en el nodo

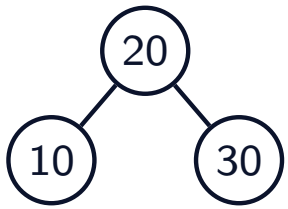
La primera rotación convierte LR en LL; la segunda es la de siempre. RL es el espejo: derecha en el hijo, izquierda en el nodo.

Un ejemplo completo: insertar $10, 20, 30, 40, 50, 25$

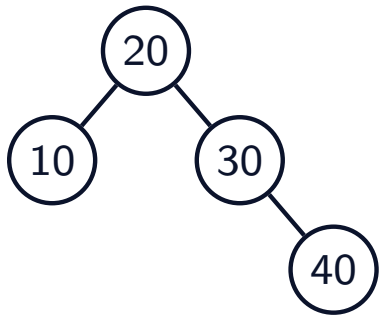


10, 20, 30: RR en 10

Un ejemplo completo: insertar $10, 20, 30, 40, 50, 25$

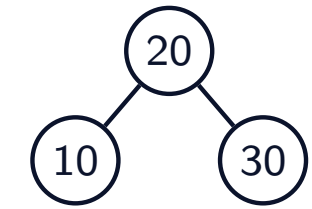


$10, 20, 30$: RR en 10

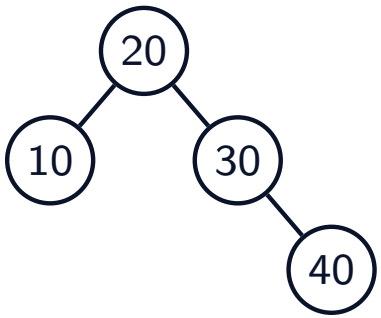


$+40$: sin rotaciones

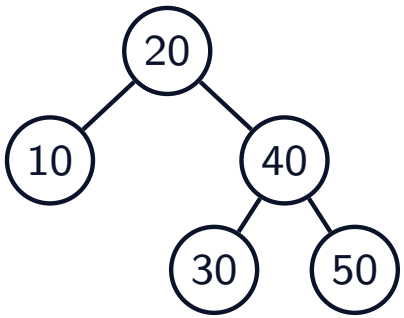
Un ejemplo completo: insertar $10, 20, 30, 40, 50, 25$



10, 20, 30: RR en 10

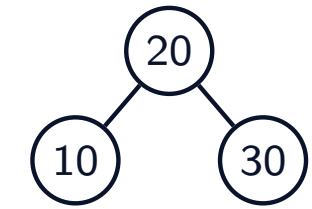


+40: sin rotaciones

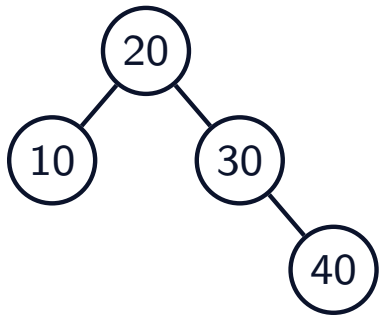


+50: RR en 30

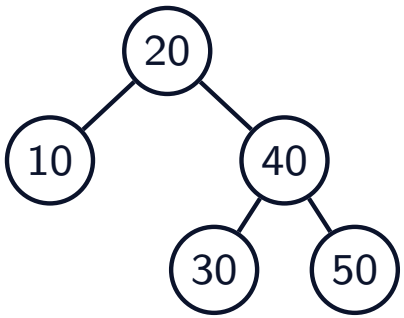
Un ejemplo completo: insertar $10, 20, 30, 40, 50, 25$



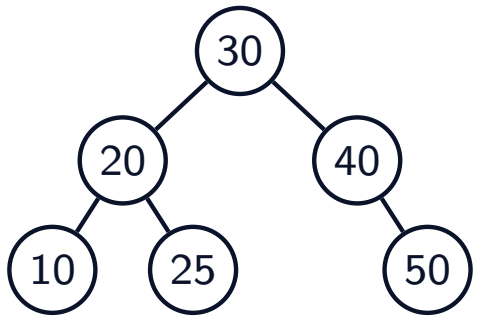
10, 20, 30: RR en 10



+40: sin rotaciones



+50: RR en 30



+25: RL en 20

Al insertar 25, el primer ancestro desbalanceado es 20 ($FB = 0 - 2 = -2$), y su hijo 40 tiene $FB = +1$: caso RL. Rotar a la derecha en 40, luego a la izquierda en 20.

¿Y al eliminar?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Eliminar como en un BST, y rebalancear subiendo

Con los mismos cuatro casos, en cada ancestro del nodo que se eliminó.

¿Y al eliminar?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Eliminar como en un BST, y rebalancear subiendo

Con los mismos cuatro casos, en cada ancestro del nodo que se eliminó.

¿Alcanza con una sola rotación, como al insertar?

¿Y al eliminar?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Eliminar como en un BST, y rebalancear subiendo

Con los mismos cuatro casos, en cada ancestro del nodo que se eliminó.

¿Alcanza con una sola rotación, como al insertar?

No siempre

Al insertar, la rotación deja el subárbol con la misma altura que tenía antes, y arriba nada cambia. Al eliminar, una rotación puede dejarlo **más bajo**, y desbalancear a un ancestro más arriba. Aun así son $O(\lg n)$ rotaciones: a lo más una por nivel.

Red-Black Tree

Otra forma de balancear: colores

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Red-Black Tree (Bayer, 1972; Guibas y Sedgewick, 1978)

Un BST donde cada nodo es **rojo** o **negro**, y:

1. La raíz es negra.
2. Las hojas NIL (los punteros nulos, vistos como nodos) son negras.
3. Un nodo rojo tiene sus dos hijos negros: no hay dos rojos seguidos.
4. Desde cualquier nodo, todos los caminos hacia las hojas NIL de abajo cruzan la **misma** cantidad de nodos negros.

Otra forma de balancear: colores

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

¿Qué tienen que ver los colores con la altura?

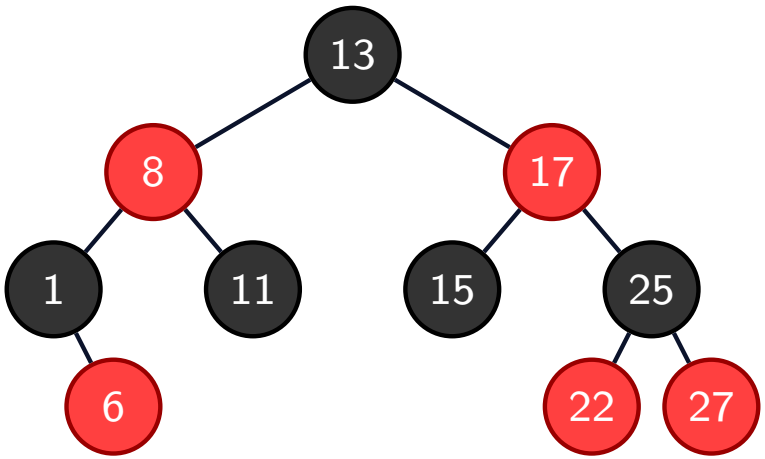
Otra forma de balancear: colores

¿Qué tienen que ver los colores con la altura?

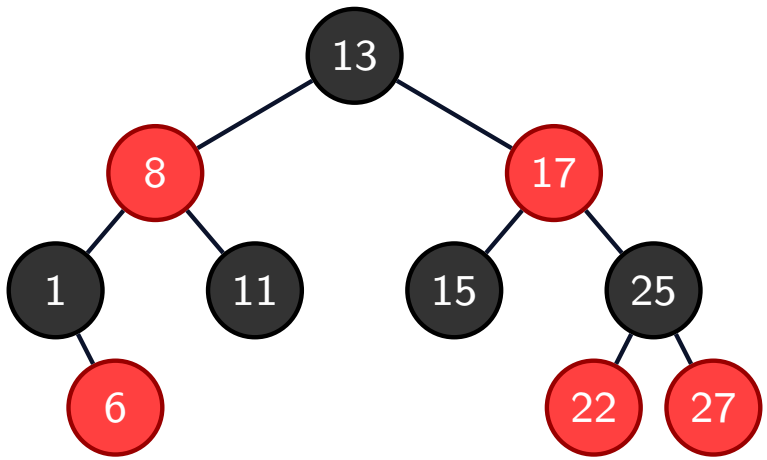
Los negros van parejos, y los rojos no se pueden amontonar

La regla 4 obliga a que todos los caminos tengan los mismos negros; la regla 3 impide estirar un camino metiéndole muchos rojos. Ningún camino puede ser más del doble de largo que otro.

Un Red-Black Tree válido



Un Red-Black Tree válido



Las hojas NIL no se dibujan. Desde 13, todo camino hacia una NIL cruza exactamente 2 nodos negros sin contar al 13: por ejemplo $13 \to 8 \to 1 \to 6 \to$ NIL cruza 1 y la NIL, y $13 \to 17 \to 25 \to 27 \to$ NIL cruza 25 y la NIL. Ningún rojo tiene un hijo rojo.

¿Por qué la altura es $O(\lg n)$?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Altura negra

$bh(x)$: la cantidad de nodos negros en cualquier camino desde x (sin contarlo) hasta una hoja NIL. Por la propiedad 4, no depende del camino.

¿Por qué la altura es $O(\lg n)$?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Altura negra

$bh(x)$: la cantidad de nodos negros en cualquier camino desde x (sin contarlo) hasta una hoja NIL. Por la propiedad 4, no depende del camino.

Lema

El subárbol de x tiene al menos $2^{bh(x)} - 1$ nodos internos (sin contar las NIL).

¿Por qué la altura es $O(\lg n)$?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Altura negra

$bh(x)$: la cantidad de nodos negros en cualquier camino desde x (sin contarlo) hasta una hoja NIL. Por la propiedad 4, no depende del camino.

Lema

El subárbol de x tiene al menos $2^{bh(x)} - 1$ nodos internos (sin contar las NIL).

Demostración, por inducción

Si x es una NIL: $bh(x) = 0$, y tiene $0 = 2^0 - 1$ nodos. Si no, cada hijo tiene altura negra $bh(x)$ (si es rojo) o $bh(x) - 1$ (si es negro), y por hipótesis al menos $2^{bh(x)-1} - 1$ nodos. Así, x tiene al menos
$2(2^{bh(x)-1} - 1) + 1 = 2^{bh(x)} - 1$. ■

¿Por qué la altura es $O(\lg n)$?

Por la propiedad 3, al menos la mitad de cada camino es negra

En un camino de la raíz a una NIL con h aristas no hay dos rojos seguidos, así que al menos la mitad de sus nodos (sin contar la raíz) son negros: $bh(\text{raíz}) \geq h/2$.

¿Por qué la altura es $O(\lg n)$?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Por la propiedad 3, al menos la mitad de cada camino es negra

En un camino de la raíz a una NIL con h aristas no hay dos rojos seguidos, así que al menos la mitad de sus nodos (sin contar la raíz) son negros: $bh(\text{raíz}) \geq h/2$.

Juntando con el lema

$n \geq 2^{bh(\text{raíz})} - 1 \geq 2^{h/2} - 1$, es decir,

$$h \leq 2 \lg(n+1).$$

¿Por qué la altura es $O(\lg n)$?

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Por la propiedad 3, al menos la mitad de cada camino es negra

En un camino de la raíz a una NIL con h aristas no hay dos rojos seguidos, así que al menos la mitad de sus nodos (sin contar la raíz) son negros: $bh(\text{raíz}) \geq h/2$.

Juntando con el lema

$n \geq 2^{bh(\text{raíz})} - 1 \geq 2^{h/2} - 1$, es decir,

$$h \leq 2 \lg(n+1).$$

En el peor caso, un Red-Black Tree es algo más alto que un AVL ($2 \lg n$ contra $1{,}45 \lg n$), pero sigue siendo $O(\lg n)$.

Insertar en un Red-Black Tree: la idea

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El nodo nuevo entra **rojo**

Así ninguna altura negra cambia: la propiedad 4 sigue intacta.

Insertar en un Red-Black Tree: la idea

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El nodo nuevo entra **rojo**

Así ninguna altura negra cambia: la propiedad 4 sigue intacta.

¿Qué puede romperse entonces?

Insertar en un Red-Black Tree: la idea

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

El nodo nuevo entra **rojo**

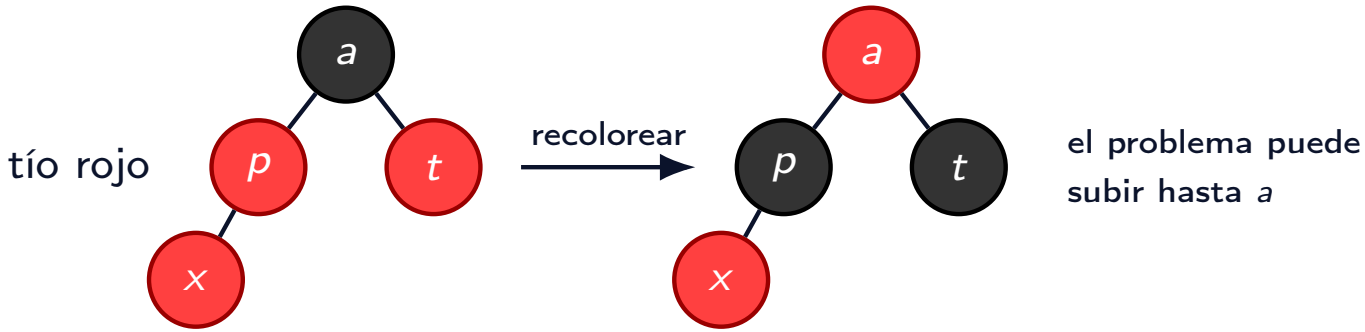
Así ninguna altura negra cambia: la propiedad 4 sigue intacta.

¿Qué puede romperse entonces?

Solo dos cosas

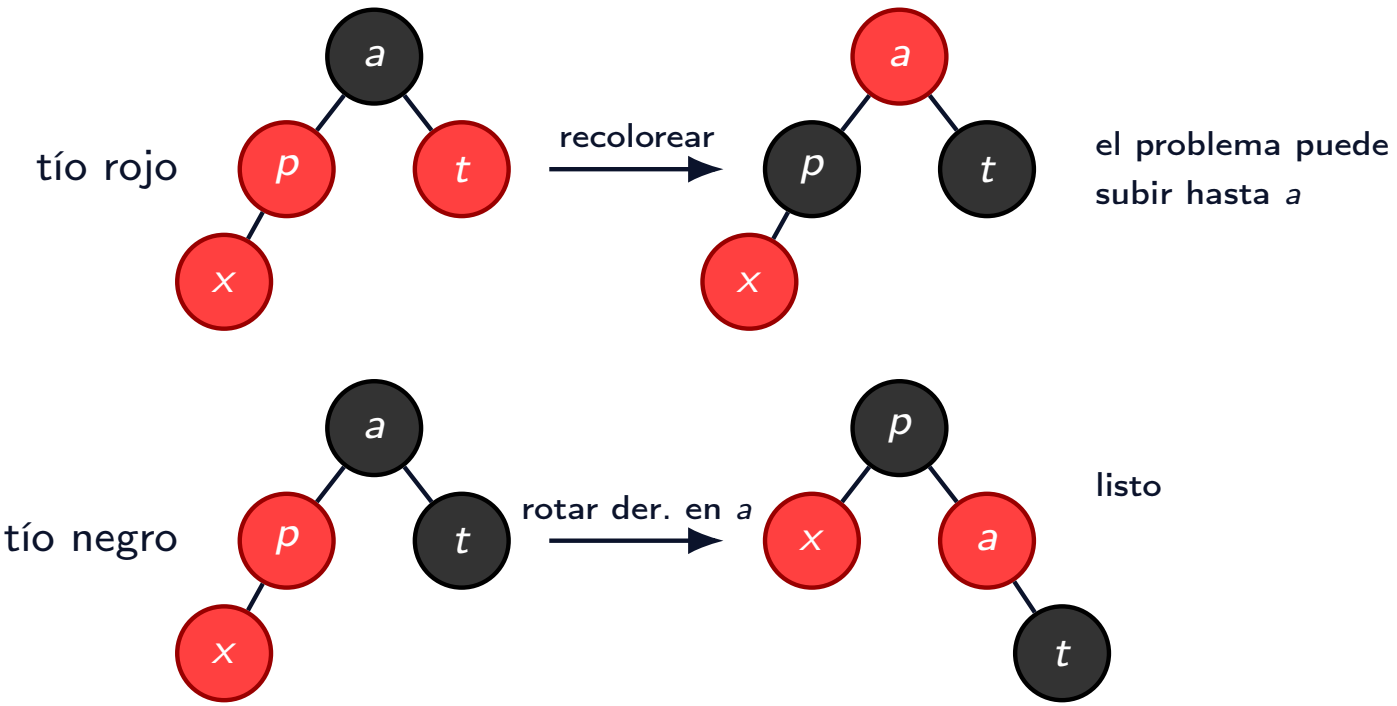
Que la raíz quede roja (si el árbol estaba vacío: se pinta de negro), o dos rojos seguidos (si el padre del nuevo es rojo). Para lo segundo, hay que mirar al **tío**: el hermano del padre.

Dos rojos seguidos: mirar al tío



x: el nuevo, p: su padre, a: su abuelo, t: su tío.

Dos rojos seguidos: mirar al tío



x: el nuevo, p: su padre, a: su abuelo, t: su tío.

Si x fuera hijo **derecho** de p, primero se rota en p para llevarlo a esta forma. En total, a lo más 2 rotaciones; el detalle, en la clase 18.

AVL vs. Red-Black

	AVL	**Red-Black**
Lo que se exige	$\lvert FB(v)\rvert \leq 1$	las propiedades de colores
Altura en el peor caso	$< 1{,}45 \lg(n+2)$	$\leq 2 \lg(n+1)$

AVL vs. Red-Black

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

	AVL	**Red-Black**
Lo que se exige	$\lvert FB(v)\rvert \leq 1$	las propiedades de colores
Altura en el peor caso	$< 1{,}45 \lg(n+2)$	$\leq 2 \lg(n+1)$
Rotaciones al insertar	a lo más 2	a lo más 2
Rotaciones al eliminar	hasta $O(\lg n)$	a lo más 3

AVL vs. Red-Black

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

	AVL	**Red-Black**
Lo que se exige	$\lvert FB(v)\rvert \leq 1$	las propiedades de colores
Altura en el peor caso	$< 1{,}45 \lg(n+2)$	$\leq 2 \lg(n+1)$
Rotaciones al insertar	a lo más 2	a lo más 2
Rotaciones al eliminar	hasta $O(\lg n)$	a lo más 3

Los dos garantizan $O(\lg n)$; cambian las constantes

El AVL es más bajo, así que busca un poco más rápido. El Red-Black modifica menos al actualizar: por eso lo usan `std::map` y `std::set` en la STL de GCC, y `TreeMap` en Java.

El TDA

El mismo TDA de un BST, ahora con garantía

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Operación	BST	BST balanceado
Buscar, Mínimo, Máximo	$O(h)$	$O(\lg n)$
Sucesor, Predecesor	$O(h)$	$O(\lg n)$
Insertar, Eliminar	$O(h)$	$O(\lg n)$
Recorrido-Inorder	$\Theta(n)$	$\Theta(n)$

El mismo TDA de un BST, ahora con garantía

Operación	BST	BST balanceado
Buscar, Mínimo, Máximo	$O(h)$	$O(\lg n)$
Sucesor, Predecesor	$O(h)$	$O(\lg n)$
Insertar, Eliminar	$O(h)$	$O(\lg n)$
Recorrido-Inorder	$\Theta(n)$	$\Theta(n)$

Las operaciones que no modifican el árbol son **exactamente** las del BST: un árbol balanceado **es** un BST. Solo Insertar y Eliminar hacen trabajo extra: rebalancear. Las próximas dos clases: cómo se implementa ese trabajo extra en un AVL y en un Red-Black.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Un BST degenera con inserciones ordenadas; exigir que sea completo cuesta $\Theta(n)$ por inserción. La salida es una condición más débil que igual garantice $h = O(\lg n)$.
- Una rotación cambia la forma en $O(1)$ sin tocar el orden inorder.

Cierre de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Un BST degenera con inserciones ordenadas; exigir que sea completo cuesta $\Theta(n)$ por inserción. La salida es una condición más débil que igual garantice $h = O(\lg n)$.
- Una rotación cambia la forma en $O(1)$ sin tocar el orden inorder.
- AVL: $|FB| \leq 1$ en todo nodo, $h < 1{,}45 \lg(n+2)$; al insertar, una o dos rotaciones en el primer ancestro desbalanceado (LL, RR, LR, RL).
- Red-Black: colores con 4 reglas, $h \leq 2 \lg(n+1)$; el nuevo entra rojo, y si queda con padre rojo se mira al tío.

Logros de la sesión

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos explicar por qué un BST necesita balanceo, y por qué "completo" es una exigencia demasiado cara.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos explicar por qué un BST necesita balanceo, y por qué "completo" es una exigencia demasiado cara.
- Sabemos rotar un subárbol y verificar que el orden se conserva.

Logros de la sesión

- Sabemos explicar por qué un BST necesita balanceo, y por qué "completo" es una exigencia demasiado cara.
- Sabemos rotar un subárbol y verificar que el orden se conserva.
- Sabemos verificar la propiedad AVL, demostrar su cota de altura y rebalancear tras una inserción.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos explicar por qué un BST necesita balanceo, y por qué "completo" es una exigencia demasiado cara.
- Sabemos rotar un subárbol y verificar que el orden se conserva.
- Sabemos verificar la propiedad AVL, demostrar su cota de altura y rebalancear tras una inserción.
- Sabemos verificar las propiedades de un Red-Black Tree y demostrar que su altura es a lo más $2 \lg(n+1)$.

Logros de la sesión

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

- Sabemos explicar por qué un BST necesita balanceo, y por qué "completo" es una exigencia demasiado cara.
- Sabemos rotar un subárbol y verificar que el orden se conserva.
- Sabemos verificar la propiedad AVL, demostrar su cota de altura y rebalancear tras una inserción.
- Sabemos verificar las propiedades de un Red-Black Tree y demostrar que su altura es a lo más $2 \lg(n+1)$.
- Sabemos comparar AVL y Red-Black.

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

Gracias